Annexe POC 2 - Exécuter un code hostile

Mini-cours · à lire avant d'attaquer le POC runtime

Le but n'est pas d'apprendre la programmation système, mais de comprendre une idée : *comment faire tourner un programme qu'on ne contrôle pas, sans qu'il puisse nuire.* Les quelques appels ci-dessous suffisent.

---

1. Le problème

Un bot est écrit par quelqu'un d'autre. Il peut planter, boucler à l'infini, dévorer la mémoire, ou simplement ne jamais répondre. S'il est exécuté **à l'intérieur de notre propre programme** (par un appel de fonction), son plantage devient le nôtre, et sa boucle infinie gèle tout.

La parade tient en une phrase : **on ne l'exécute pas chez soi, on l'exécute à côté, dans un process séparé que l'on surveille de loin.** Un process isolé peut planter sans entraîner le nôtre, et on peut l'arrêter dès qu'il dépasse les bornes.

---

2. Un process séparé : `fork` + `exec`

`fork()` duplique le process courant. À partir de là, deux process identiques s'exécutent. On les distingue par la valeur de retour :

```
pid_t pid = fork();
if (pid == 0) {
    // ENFANT : destiné à devenir le bot
} else {
    // PARENT : reste le programme principal, pid = numéro de l'enfant
}
```

Dans l'enfant, `execvp()` **remplace** le programme courant par un autre - le bot. L'enfant cesse d'être une copie du parent et devient le bot :

```
execvp("./bot_ok", args);   // après cet appel, l'enfant EST bot_ok
```

Le bot s'exécute alors dans son propre process. S'il plante, c'est lui qui tombe, pas le programme principal.

---

3. Lire sa sortie : un tube (`pipe`)

Le bot imprime sur sa sortie standard. Pour lire ce qu'il imprime, on crée un **tube** (`pipe`) *avant* le `fork` , et on branche la sortie de l'enfant dessus avec `dup2` :

```
int out[2];          // out[0] = lecture (parent), out[1] = écriture (bot)
pipe(out);
// ... fork ...
// dans l'ENFANT :
dup2(out[1], STDOUT_FILENO);   // ce que le bot imprime part dans le tube
```

Le parent lit ensuite sur `out[0]` ce que le bot écrit. Un seul tube suffit ici : on lit le bot, on ne lui envoie rien.

---

4. Lire, mais avec un délai maximal

Lire directement avec `read` est **bloquant** : si le bot n'imprime rien, `read` attend indéfiniment, et le programme est figé avec lui. C'est précisément le cas `bot_loop`.

La solution : ne jamais attendre sans limite. `poll()` attend qu'il y ait quelque chose à lire, **mais au plus un délai donné** :

```
struct pollfd pfd = { .fd = out[0], .events = POLLIN };
int ready = poll(&pfd, 1, 500);   // attend au plus 500 ms
```

Trois issues :

- `ready > 0` : le bot a imprimé quelque chose → verdict `OK` ;
- `ready == 0` : 500 ms sans rien → verdict `TIMEOUT`, il faut l'arrêter ;
- `ready < 0` : erreur.

---

5. L'arrêter : `SIGKILL`

Pour arrêter un process, on lui envoie un **signal**. Deux signaux signifient « arrête-toi » :

- `SIGTERM` : « arrête-toi, s'il te plaît ». Un process **peut l'ignorer**.
- `SIGKILL` : « arrête-toi, maintenant ». **Impossible à ignorer**, le noyau met fin au process.

Face à un code hostile, une demande polie n'a pas de sens : un bot malveillant intercepterait `SIGTERM` et poursuivrait. On utilise donc `SIGKILL` :

```
kill(pid, SIGKILL);
```

---

6. Nettoyer derrière lui : `waitpid`

Un process terminé n'a pas tout à fait disparu : il laisse une trace (son code de sortie) que le parent doit venir récupérer. Tant que ce n'est pas fait, il demeure dans un état **zombie** - terminé, mais toujours présent dans la table des process. Un runtime qui lance des milliers de bots sans les récupérer finit par saturer le système.

```
waitpid(pid, &status, 0);   // récupère l'enfant terminé et le fait disparaître
```

On peut surveiller les process actuellement existants avec `ps`.

---

7. Pour aller plus loin (dans le lot, pas dans le POC)

Arrêter un bot lent ne protège pas d'un bot qui dévore la mémoire ou le CPU. Le vrai runtime ajoutera des **limites de ressources** (`setrlimit`) posées dans l'enfant juste avant l' `exec` : plafond de mémoire, plafond de temps CPU. Le noyau met alors lui-même fin au process qui dépasse. Ce n'est pas demandé dans le POC, mais c'est l'horizon du lot : **le process isolé n'est pas seulement surveillé de l'extérieur, il est enfermé dans une cage dont il ne peut pas sortir.**

En une phrase

Exécuter un code hostile, c'est : **le placer à côté** (`fork` / `exec`), **lui parler par un tube** (`pipe` / `dup2`), **ne jamais l'attendre sans limite** (`poll`), **pouvoir l'arrêter de force** (`SIGKILL`), **et nettoyer derrière** (`waitpid`).